

```
import pandas as pd
import numpy as np
import statsmodels.api as sm
```

```
df = pd.read_csv('/Bootcamp/Real estate.csv')
```

```
df.head()
```



	No	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
0	1	2012.917	32.0	84.87882	10	24.98298	121.54024	37.9
1	2	2012.917	19.5	306.59470	9	24.98034	121.53951	42.2
2	3	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3
3	4	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3

```
X = df['X2 house age']
y = df['Y house price of unit area']
```

```
X = sm.add_constant(X)
```

```
model = sm.OLS(y, X).fit()
```

```
model.summary()
```



OLS Regression Results

Dep. Variable:	Y house price of unit area	R-squared:	0.044
Model:	OLS	Adj. R-squared:	0.042
Method:	Least Squares	F-statistic:	19.11
Date:	Fri, 13 Sep 2024	Prob (F-statistic):	1.56e-05
Time:	12:14:25	Log-Likelihood:	-1658.3
No. Observations:	414	AIC:	3321.
Df Residuals:	412	BIC:	3329.
Df Model:	1		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025	0.975]
const	42.4347	1.211	35.042	0.000	40.054	44.815
X2 house age	-0.2515	0.058	-4.372	0.000	-0.365	-0.138

Omnibus: 48.404 Durbin-Watson: 1.957
 Prob(Omnibus): 0.000 Jarque-Bera (JB): 119.054
 Skew: 0.589 Prob(JB): 1.40e-26
 Kurtosis: 5.348 Cond. No. 39.0

Notes:

```
X = df[['X2 house age', 'X3 distance to the nearest MRT station' ]]
y = df['Y house price of unit area']
```

```
X = sm.add_constant(X)
```

```
model2 = sm.OLS(y, X).fit()
```

```
model2.summary()
```



OLS Regression Results									
Dep. Variable:		Y house price of unit area			R-squared:		0.491		
Model:		OLS			Adj. R-squared:		0.489		
Method:		Least Squares			F-statistic:		198.3		
Date:		Fri, 13 Sep 2024			Prob (F-statistic):		5.07e-61		
Time:		12:17:20			Log-Likelihood:		-1527.9		
No. Observations:		414			AIC:		3062.		
Df Residuals:		411			BIC:		3074.		
Df Model:		2							
Covariance Type: nonrobust									
		coef	std err	t	P> t	[0.025	0.975]		
const		49.8856	0.968	51.547	0.000	47.983	51.788		
X2 house age		-0.2310	0.042	-5.496	0.000	-0.314	-0.148		
X3 distance to the nearest MRT station		-0.0072	0.000	-18.997	0.000	-0.008	-0.006		
Omnibus:		161.397	Durbin-Watson:		2.130				
Prob(Omnibus):		0.000	Jarque-Bera (JB):		1297.792				
Skew:		1.443	Prob(JB):		1.54e-282				
Kurtosis:		11.180	Cond. No.		3.37e+03				

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.37e+03. This might indicate that there are

 **Shikder Rasel**
Sep 13, 2024



All Okay, no need to change my data type

df.dtypes



No	int64
X1 transaction date	float64
X2 house age	float64
X3 distance to the nearest MRT station	float64
X4 number of convenience stores	int64
X5 latitude	float64
X6 longitude	float64
Y house price of unit area	float64
dtype: object	

```
X = df.drop(['X1 transaction date', 'Y house price of unit area'], axis=1)
y = df['Y house price of unit area']
```

```
X = sm.add_constant(X)
```

```
model3 = sm.OLS(y, X).fit()
```

```
model3.summary()
```

 **Shikder Rasel**
Sep 13, 2024



95% R square is so good, after dropping 2 variables.



OLS Regression Results

Dep. Variable: Y house price of unit area **R-squared (uncentered):** 0.951
Model: OLS **Adj. R-squared (uncentered):** 0.951
Method: Least Squares **F-statistic:** 1331.
Date: Fri, 13 Sep 2024 **Prob (F-statistic):** 2.19e-264
Time: 12:21:52 **Log-Likelihood:** -1492.1
No. Observations: 414 **AIC:** 2996.
Df Residuals: 408 **BIC:** 3020.
Df Model: 6
Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
No	-0.0042	0.004	-1.150	0.251	-0.011	0.003
X2 house age	-0.2708	0.039	-6.948	0.000	-0.347	-0.194
X3 distance to the nearest MRT station	-0.0047	0.000	-10.336	0.000	-0.006	-0.004
X4 number of convenience stores	1.1530	0.190	6.069	0.000	0.780	1.527
X5 latitude	226.6778	42.875	5.287	0.000	142.394	310.962
X6 longitude	-46.2085	8.809	-5.246	0.000	-63.524	-28.893

Omnibus: 238.989 **Durbin-Watson:** 2.151
Prob(Omnibus): 0.000 **Jarque-Bera (JB):** 3746.475
Skew: 2.114 **Prob(JB):** 0.00
Kurtosis: 17.118 **Cond. No.** 1.66e+05

Notes:

- [1] R^2 is computed without centering (uncentered) since the model does not contain a constant.
 [2] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [3] The condition number is large, 1.66e+05. This might indicate that there are

```
X = df.drop(['Y house price of unit area'], axis=1)
y = df['Y house price of unit area']
```

```
X = sm.add_constant(X)
```

```
model4 = sm.OLS(y, X).fit()
```

```
model4.summary()
```



OLS Regression Results

Dep. Variable: Y house price of unit area **R-squared:** 0.583
Model: OLS **Adj. R-squared:** 0.576
Method: Least Squares **F-statistic:** 81.21
Date: Fri, 13 Sep 2024 **Prob (F-statistic):** 3.06e-73
Time: 12:23:59 **Log-Likelihood:** -1486.5
No. Observations: 414 **AIC:** 2989.
Df Residuals: 406 **BIC:** 3021.
Df Model: 7
Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	-1.404e+04	6788.074	-2.068	0.039	-2.74e+04	-693.919
No	-0.0036	0.004	-0.984	0.326	-0.011	0.004
X1 transaction date	5.0791	1.559	3.259	0.001	2.015	8.143
X2 house age	-0.2708	0.039	-7.026	0.000	-0.347	-0.195
X3 distance to the nearest MRT station	-0.0045	0.001	-6.289	0.000	-0.006	-0.003
X4 number of convenience stores	1.1293	0.188	6.000	0.000	0.759	1.499
X5 latitude	224.6729	44.576	5.040	0.000	137.045	312.301
X6 longitude	-14.4236	48.626	-0.297	0.767	-110.015	81.168

Omnibus: 234.150 **Durbin-Watson:** 2.158
Prob(Omnibus): 0.000 **Jarque-Bera (JB):** 3732.009
Skew: 2.047 **Prob(JB):** 0.00
Kurtosis: 17.127 **Cond. No.** 3.73e+07

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 3.73e+07. This might indicate that there are



Shikder Rasel
Sep 13, 2024



It just shows that Model3 was better dropping 2 variables rather 1



Shikder Rasel
Sep 13, 2024



This confirms that model3 is the best fit for highest R square output

Double-click (or enter) to edit

```
X = df.drop(['X1 transaction date', 'X6 longitude', 'Y house price of unit area'], axis=1)
y = df['Y house price of unit area']
```

```
X = sm.add_constant(X)
```

```
model5 = sm.OLS(y, X).fit()
```

```
model5.summary()
```

OLS Regression Results

Dep. Variable:	Y house price of unit area	R-squared:	0.572
Model:	OLS	Adj. R-squared:	0.567
Method:	Least Squares	F-statistic:	109.2
Date:	Fri, 13 Sep 2024	Prob (F-statistic):	5.14e-73
Time:	12:27:28	Log-Likelihood:	-1491.8
No. Observations:	414	AIC:	2996.
Df Residuals:	408	BIC:	3020.
Df Model:	5		

Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	-5894.3613	1112.595	-5.298	0.000	-8081.495	-3707.227
No	-0.0041	0.004	-1.108	0.269	-0.011	0.003
X2 house age	-0.2700	0.039	-6.934	0.000	-0.347	-0.193
X3 distance to the nearest MRT station	-0.0042	0.000	-8.504	0.000	-0.005	-0.003
X4 number of convenience stores	1.1603	0.190	6.118	0.000	0.787	1.533
X5 latitude	237.8051	44.555	5.337	0.000	150.219	325.391

Omnibus: 243.632 **Durbin-Watson:** 2.157

Prob(Omnibus): 0.000 **Jarque-Bera (JB):** 3937.832

Skew: 2.160 **Prob(JB):** 0.00

Kurtosis: 17.478 **Cond. No.** 4.22e+06

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.22e+06. This might indicate that there are

Shikder Rasel
Sep 13, 2024

Still Model3 gave the most output

```
df.head()
```

	No	X1 transaction date	X2 house age	X3 distance to the nearest MRT station	X4 number of convenience stores	X5 latitude	X6 longitude	Y house price of unit area
0	1	2012.917	32.0	84.87882	10	24.98298	121.54024	37.9
1	2	2012.917	19.5	306.59470	9	24.98034	121.53951	42.2
2	3	2013.583	13.3	561.98450	5	24.98746	121.54391	47.3
3	4	2013.583	18.8	531.88158	5	24.98718	121.54391	51.8

```
X = df.drop(['No', 'X1 transaction date', 'Y house price of unit area'], axis=1)
y = df['Y house price of unit area']
```

```
X = sm.add_constant(X)
```

```
model6 = sm.OLS(y, X).fit()
```

```
model6.summary()
```



OLS Regression Results

Dep. Variable: Y house price of unit area **R-squared:** 0.571
Model: OLS **Adj. R-squared:** 0.566
Method: Least Squares **F-statistic:** 108.7
Date: Fri, 13 Sep 2024 **Prob (F-statistic):** 9.34e-73
Time: 12:40:58 **Log-Likelihood:** -1492.4
No. Observations: 414 **AIC:** 2997.
Df Residuals: 408 **BIC:** 3021.
Df Model: 5
Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
const	-4945.5951	6211.157	-0.796	0.426	-1.72e+04	7264.269
X2 house age	-0.2689	0.039	-6.896	0.000	-0.346	-0.192
X3 distance to the nearest MRT station	-0.0043	0.001	-5.888	0.000	-0.006	-0.003
X4 number of convenience stores	1.1630	0.190	6.114	0.000	0.789	1.537
X5 latitude	237.7672	44.948	5.290	0.000	149.409	326.126
X6 longitude	-7.8055	49.149	-0.159	0.874	-104.422	88.811

Omnibus: 240.068 **Durbin-Watson:** 2.149
Prob(Omnibus): 0.000 **Jarque-Bera (JB):** 3748.747
Skew: 2.129 **Prob(JB):** 0.00
Kurtosis: 17.114 **Cond. No.** 2.35e+07

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large. 2.35e+07. This might indicate that there are

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
X = df.drop(['No', 'X1 transaction date', 'Y house price of unit area'], axis=1)
y = df['Y house price of unit area']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model7 = LinearRegression()
model7.fit(X_train, y_train)
```

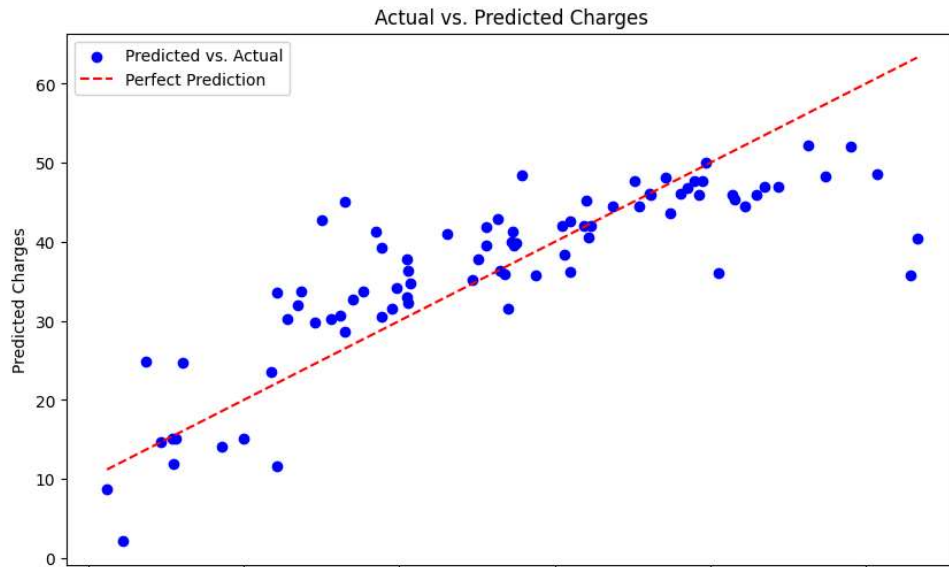


```
LinearRegression()
LinearRegression()
```

```
y_pred = model7.predict(X_test)
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, color='blue', label='Predicted vs. Actual')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle='--', label='Actual vs. Predicted')
plt.xlabel('Actual Charges')
plt.ylabel('Predicted Charges')
plt.title('Actual vs. Predicted Charges')
plt.legend()
plt.show()
```



```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
# Print performance metrics
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"Mean Absolute Error (MAE): {mae}")
print(f"R-squared (R²): {r2}")
```

```
Mean Squared Error (MSE): 54.58094520086248
Root Mean Squared Error (RMSE): 7.387891796775484
Mean Absolute Error (MAE): 5.350138374356204
R-squared (R²): 0.6746481382828156
```

```
X = df.drop(['X1 transaction date', 'Y house price of unit area'], axis=1)
y = df['Y house price of unit area']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model8 = LinearRegression()
model8.fit(X_train, y_train)
```

```
LinearRegression()
LinearRegression()
```

```
y_pred = model8.predict(X_test)
```

```
import matplotlib.pyplot as plt
```

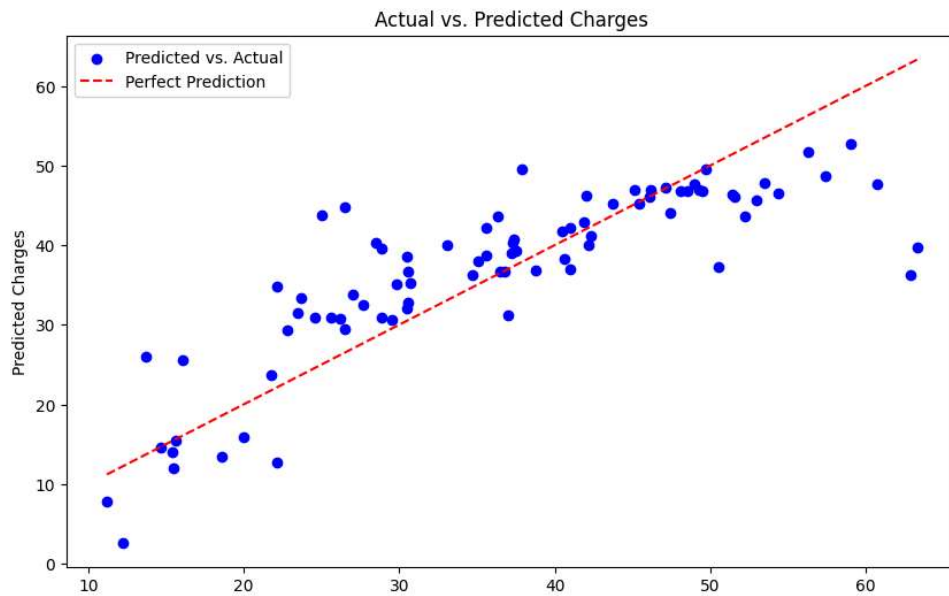
```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, color='blue', label='Predicted vs. Actual')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle='--', label='Perfect Prediction')
plt.xlabel('Actual Charges')
plt.ylabel('Predicted Charges')
plt.title('Actual vs. Predicted Charges')
plt.legend()
plt.show()
```



Shikder Rasel
Sep 13, 2024



Lower than model 7



```
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
```

```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

```
# Print performance metrics
print(f"Mean Squared Error (MSE): {mse}")
print(f"Root Mean Squared Error (RMSE): {rmse}")
print(f"Mean Absolute Error (MAE): {mae}")
print(f"R-squared (R²): {r2}")
```

```
Mean Squared Error (MSE): 55.42976176163538
Root Mean Squared Error (RMSE): 7.445116638551433
Mean Absolute Error (MAE): 5.385270702159246
R-squared (R²): 0.6695884228951919
```



Shikder Rasel
Sep 13, 2024



Model 7 is better in Sklearn library model

```
X = df.drop(['Y house price of unit area'], axis=1)
y = df['Y house price of unit area']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
model9 = LinearRegression()
model9.fit(X_train, y_train)
```



```
LinearRegression()
LinearRegression()
```

```
y_pred = model9.predict(X_test)
```

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(10, 6))
plt.scatter(y_test, y_pred, color='blue', label='Predicted vs. Actual')
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle='--', label='Perfect Prediction')
plt.xlabel('Actual Charges')
plt.ylabel('Predicted Charges')
plt.title('Actual vs. Predicted Charges')
plt.legend()
plt.show()
```